# SMARTSHELF - REPORT

## AI-Powered Replenishment Assistant

AI CAPSTONE PROJECT – SKEMA – 2025/2026

ZEJLI Mohamed-Nadir – BOUMALHAB Chrisotphe – BOUDRAA Ahmed Noureddine

# Table of Contents

Table of Contents ..... 1
1. Executive Summary ..... 2
2. Business Problem and Project Motivation ..... 2
3. Dataset and Pilot Scope ..... 2
4. Data Preparation and Cleaning ..... 3
5. Feature Engineering ..... 4
6. Modeling Strategy and Baseline ..... 4
7. Forecasting Results ..... 5
8. Cost Model and Business Value ..... 6
9. Prototype and Application ..... 6
10. Engineering and reproducibility ..... 7
11. Limitations and Future Work ..... 7
12. Conclusion ..... 8
Sources ..... 8
Table of figures ..... 8

# 1. Executive Summary

SmartShelf is an AI-powered replenishment assistant designed to support retail store managers in daily ordering decisions. The project addresses a common retail problem: deciding what to reorder and in what quantity in order to reduce both overstock and stockout situations. To do so, SmartShelf combines demand forecasting, order recommendation, and cost simulation in a single system based on the M5 Walmart dataset. The pilot focuses on three stores and the top 100 products by sales volume, which makes the project computationally manageable while remaining representative of real retail operations.

The forecasting component uses a LightGBM model with a Tweedie objective and is compared with a naive baseline based on the rolling 28-day average of past sales. Beyond technical accuracy, the project evaluates business value by translating forecast errors into overstock and stockout costs expressed in euros. On the 56-day evaluation period, SmartShelf reduces simulated inventory cost from €117,302 for the naive baseline to €35,559, which corresponds to savings of €81,743 and a 69.7% cost reduction.

# 2. Business Problem and Project Motivation

Retail replenishment is still often handled using intuition, simple spreadsheet rules, or recent historical averages. Such approaches ignore important drivers of demand such as seasonality, prices, promotions, events, and SNAP days, which can lead to costly overstock and stockout situations. SmartShelf was developed to replace this reactive logic with a more informed and data-driven approach. Rather than acting as a forecasting model only, it is designed as a decision-support prototype that connects demand prediction, order recommendation, and measurable business impact

# 3. Dataset and Pilot Scope

The project is based on the M5 Forecasting dataset, which contains real daily Walmart sales data together with calendar information and weekly selling prices. For this project, three files were used: sales_train_evaluation.csv for historical sales, calendar.csv for dates and event-related variables, and sell_prices.csv for item prices by store and week. These three sources were merged to build the final modeling dataset.

Because the full M5 dataset is very large, the project was scoped as a representative pilot. The selected scope includes three stores — CA_1, TX_1, and WI_1 — and the top 100 products by total sales volume. This results in a dataset of about 582,000 rows covering the period from 2011 to 2016. This choice makes the pipeline feasible on a standard laptop while preserving a realistic retail setting and a structure that can be scaled later.

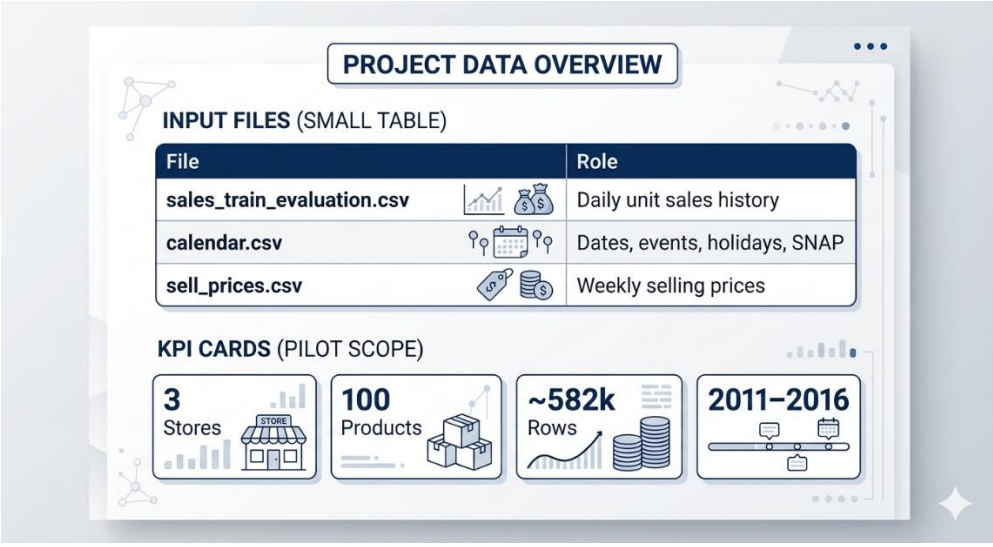


*Figure 1: Project data overview*

# 4. Data Preparation and Cleaning

The raw M5 data required several preprocessing steps before modeling. First, the sales data was filtered to the three selected stores and the top 100 products by total sales volume. Because the original sales file is stored in a wide format with one column per day, it was reshaped into a long format with one row per product, store, and date. The resulting table was then merged with the calendar data and the weekly price data to create a unified daily dataset.

Two main data issues had to be addressed. The first was missing price values, since some products are not listed every week in sell_prices.csv. To handle this, prices were forward-filled and backward-filled within each item-store group. The second was the warm-up period introduced by lag and rolling features, which creates missing values at the beginning of each time series. These initial rows were removed before training. The cleaned dataset was then saved in parquet format to make the pipeline faster and more reproducible. (because the laptop we tested in has 8gb of RAM :/ )

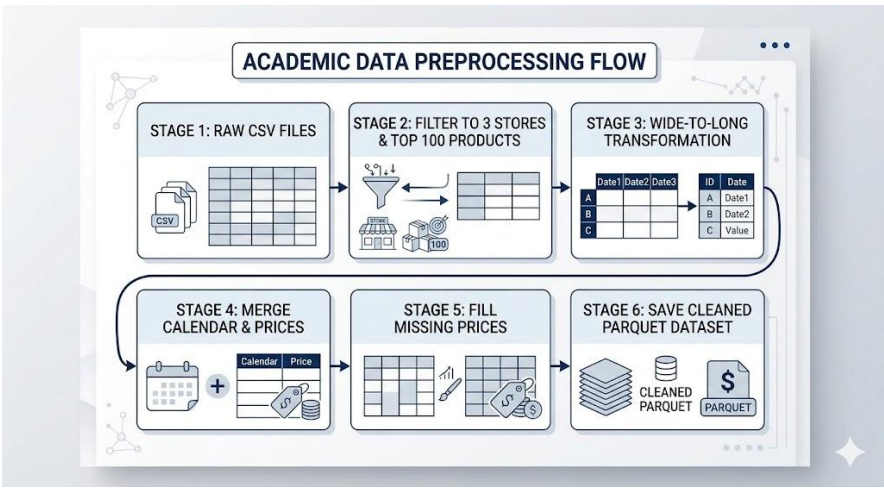


*Figure 2: Data processing flow*

# 5. Feature Engineering

To improve forecasting performance, feature engineering was applied to the merged daily dataset. The objective was to move beyond raw sales values and provide the model with information about recent demand history, short-term trends, price effects, calendar patterns, and store-product context.

The engineered features were grouped into five main families: historical demand, trend and volatility, price signals, calendar and event information, and product-store identifiers. Together, these features allow the model to capture recurring demand patterns, short-term variability, commercial effects, and contextual differences between stores and products.

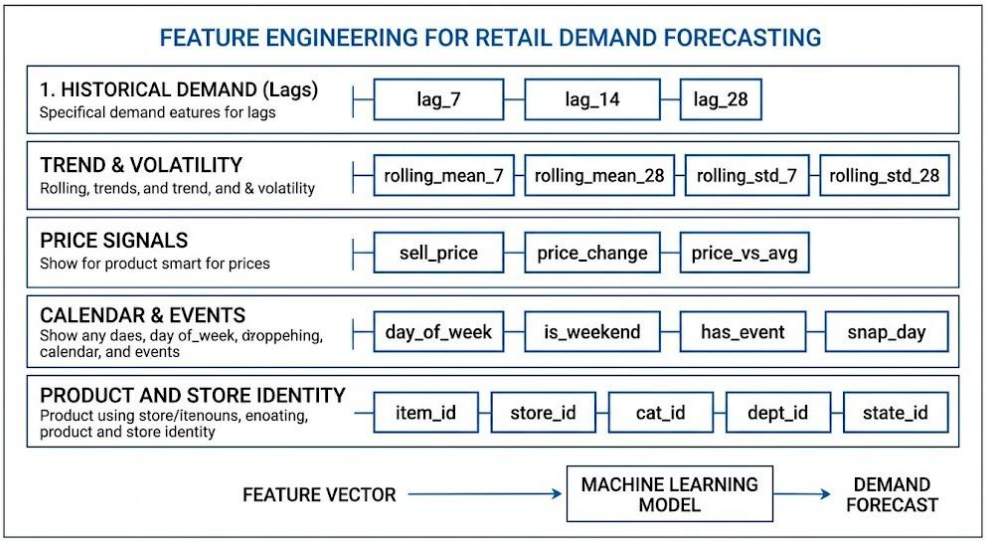


*Figure 3: Feature Engineering*

# 6. Modeling Strategy and Baseline

The forecasting component of SmartShelf is based on LightGBM, trained on the engineered feature set using a Tweedie objective. This model was selected because it is efficient on large tabular datasets and better suited to intermittent retail demand than standard regression approaches. The data was split chronologically, with observations up to 27 March 2016 used for training and the following 56 days used for validation. This preserves the time-series nature of the problem and avoids information leakage from future periods.

To evaluate the model fairly, its performance was compared with a naive baseline based on the rolling 28-day average of past sales. This baseline represents a simple but realistic replenishment rule: ordering according to what the product sold on average during the previous month. The comparison is important because the goal of SmartShelf is not only to build a forecasting model, but to show that it improves on a decision logic that could realistically be used in stores.

**COMPARISON FOR RETAIL FORECASTING REPORT**

| FORECASTING APPROACH: LIGHTGBM (TWEEDIE) | FORECASTING BASELINE: NAIVE 28-DAY ROLLING AVERAGE |
|---|---|
| • **Algorithm:** LightGBM<br>• **Objective Function:** Tweedie<br>• **Performance:** Fast processing on tabular data<br>• **Application:** Optimized for intermittent demand patterns | • **Method:** Naive (Rolling 28-Day Avg)<br>• **Operation:** Simple store manager reorder rule<br>• **Function:** Establishes baseline benchmark<br>• **Objective:** The standard to outperform |

*Figure 4: Models comparison*

# 7. Forecasting Results

The forecasting performance of SmartShelf was evaluated on a 56-day validation period and compared with the naive baseline. SmartShelf achieved a MAE of 0.9833 and an RMSE of 1.9667, while the naive baseline reached a MAE of 1.0168 and an RMSE of 2.1030. These results show that the proposed model provides a more accurate estimate of daily demand at product-store level.

Although the gain in forecasting accuracy is moderate, it is still important because even small improvements in daily retail prediction can lead to better replenishment decisions when applied across many items and stores.

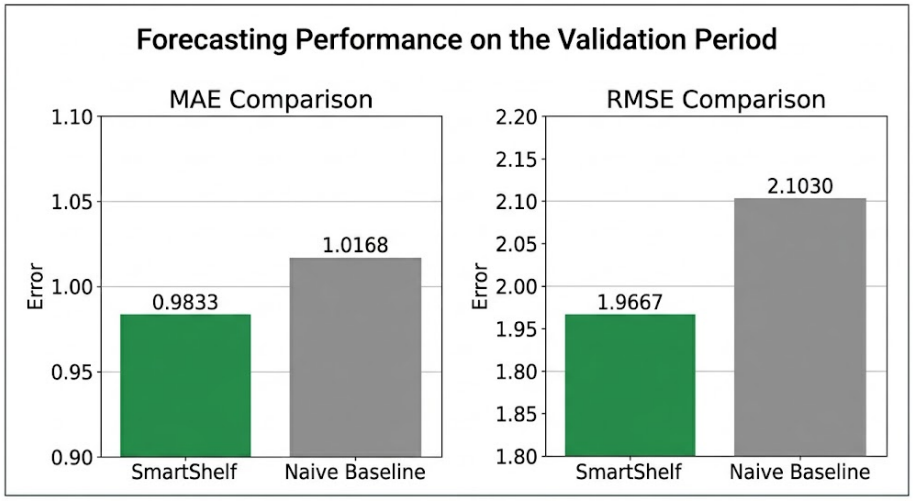


*Figure 5: RMSE + MAE comparison*

# 8. Cost Model and Business Value

A key contribution of SmartShelf is that forecasting performance is not evaluated only through MAE and RMSE. Forecast errors are also translated into economic impact through a cost simulation. When the model overestimates demand, the excess units generate overstock cost. When it underestimates demand, the missing units generate stockout cost. This makes it possible to assess whether a forecasting improvement is meaningful from a retail operations perspective.

The simulation uses literature-based assumptions rather than arbitrary values. Holding cost is set to 0.068% of item value per day, equivalent to about 25% per year, based on Silver, Pyke and Thomas. Stockout cost is set to 75% of item value, based on ECR Europe. Under these assumptions, the naive baseline generates a total simulated cost of €117,302 over the 56-day validation period, while SmartShelf reduces this cost to €35,559. This corresponds to €81,743 saved, or a 69.7% cost reduction.

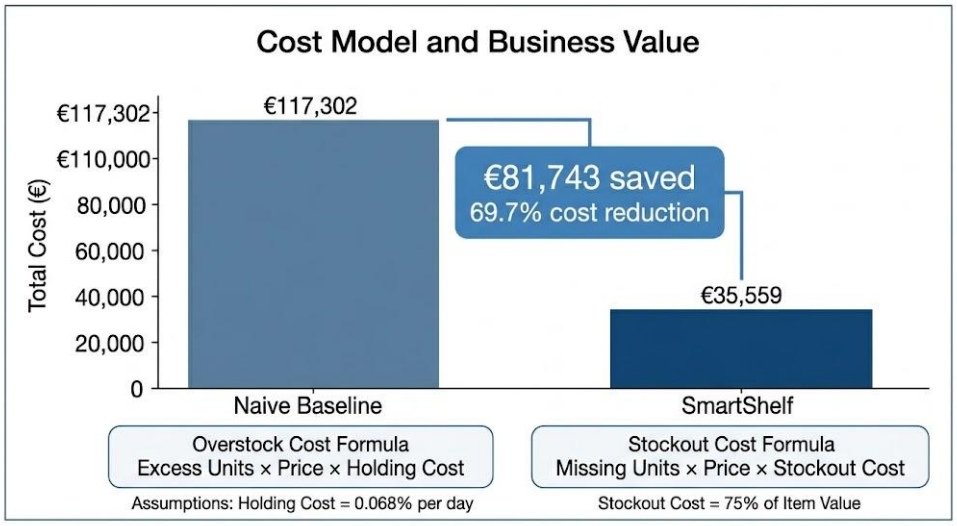


*Figure 6: Cost model and business value*

# 9. Prototype and Application

The final output of the project is a functional Streamlit application that translates the forecasting pipeline into a usable decision-support tool. The prototype is organized into three pages: Order Assistant, Cost Dashboard, and Model Insights. This structure was designed to cover three complementary needs: operational decision-making, business evaluation, and technical transparency.

The Order Assistant helps the user select a store and a product, enter the current stock level, and obtain a recommended order quantity together with a 7-day forecast and a short explanation of the recommendation. The Cost Dashboard compares SmartShelf with the naive baseline and visualizes total cost, savings by store, savings by category, and daily cost evolution. The Model Insights page presents forecasting metrics and feature importance to make the model behavior easier to interpret. As a result, the prototype connects prediction, operational action, and business value in a single interface.

# 10. Engineering and reproducibility

SmartShelf was implemented as a modular Python. The code is organized into separate components for data loading, feature engineering, model training, prediction, cost simulation, and application deployment. This structure makes the project easier to understand, maintain, and reuse. In addition, intermediate datasets are saved in parquet format, while the trained model, encoders, and metrics are saved as reusable artifacts.

To improve reproducibility, the project can be executed end to end through run_pipeline.py, which runs the four main stages of the pipeline: loading and filtering the data, engineering features, training the model, and simulating the business cost. The final system is then exposed through a Streamlit application that provides operational, business, and technical views of the project.

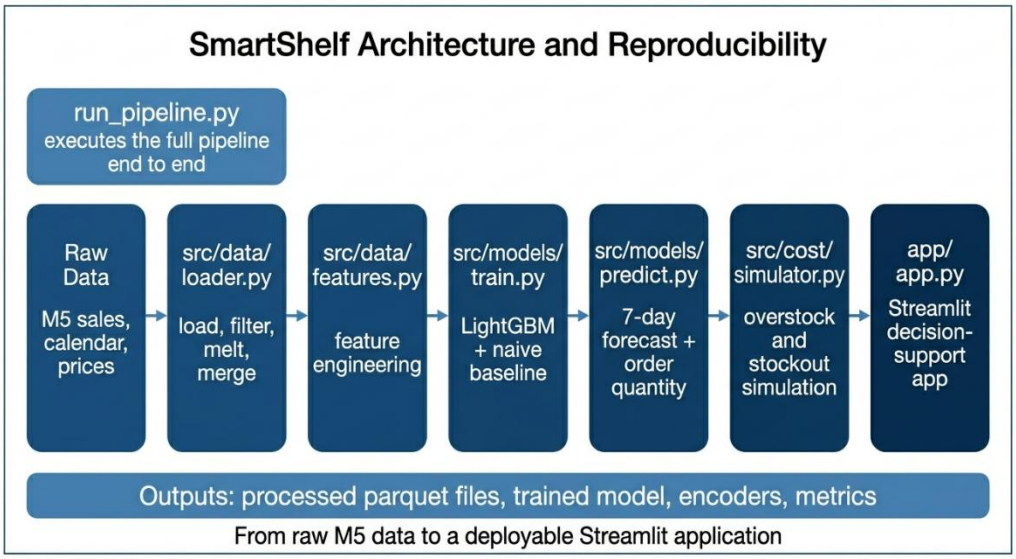


*Figure 7: Architecture and reproducibility*

# 11. Limitations and Future Work

SmartShelf was developed as a representative pilot, not as a full production retail system. Several limitations should therefore be noted. First, the cost simulation relies on fixed holding and stockout assumptions, while real cost structures vary by retailer, category, and product type. Second, the model is trained on historical data from 2011 to 2016, so more recent demand shifts would require retraining. Third, the pilot scope is limited to three stores and the top 100 products, which makes the system efficient and demonstrable, but does not yet cover the full assortment.

Future work would focus on extending both realism and scale. The first improvement would be to expand the system to more stores and products. The second would be to introduce richer business assumptions, such as category-specific cost parameters and real stock integration. Additional work could also improve interpretability through techniques such as SHAP explanations and strengthen the operational side of the tool through integration with store or POS systems.

# 12. Conclusion

SmartShelf demonstrates how machine learning can be used to support retail replenishment decisions in a practical and measurable way. Using the M5 Walmart dataset, the project combines demand forecasting, order recommendation, and cost simulation in a single pipeline. Compared with a realistic naive baseline, SmartShelf improves forecasting performance and reduces simulated inventory cost from €117,302 to €35,559, corresponding to €81,743 saved and a 69.7% reduction over the evaluation period.

Beyond forecasting alone, the project shows the value of connecting technical performance to operational and financial outcomes. The final Streamlit prototype translates model outputs into daily recommendations, business dashboards, and model insights, making the system understandable for both managers and technical users. In this sense, SmartShelf should be seen not only as a predictive model, but as a decision-support prototype that illustrates how AI can improve replenishment processes in retail.

# Sources

Silver, E. A., Pyke, D. F., & Thomas, D. J. (1998). Inventory and Production Management in Supply Chains (4th ed.).

ECR Europe. (2003). The Optimal Shelf Availability: Category Management Best Practices Report.

# Table of figures

Figure 1: Project data overview ..... 3
Figure 2: Data processing flow ..... 3
Figure 3: Feature Engineering ..... 4
Figure 4: Models comparison ..... 5
Figure 5: RMSE + MAE comparison ..... 5
Figure 6: Cost model and business value ..... 6
Figure 7: Architecture and reproducibility ..... 7